trees, using a set of easier and cleaner algorithms (is comparison to red-black trees and alike).

As we have seen in the introduction to this chapter, plain BSTs as well suffer this same problem, having their structure depend on the order in which elements are inserted.

And if we go back to last section, we saw treaps can be skewed if the particular combination of keys and priorities, and the order in which elements are inserted, is particularly unlucky, because rotations can cause the tree to get even more unbalanced (see, for instance, figure 3.9).

So, here is the idea: we can use rotations to rebalance the tree. If we strip priorities from their meaning (in our example, forget about the units in stock for each product), we can, in theory, update the priority value of each node so that fixing the heap's invariants will produce a more balanced tree.

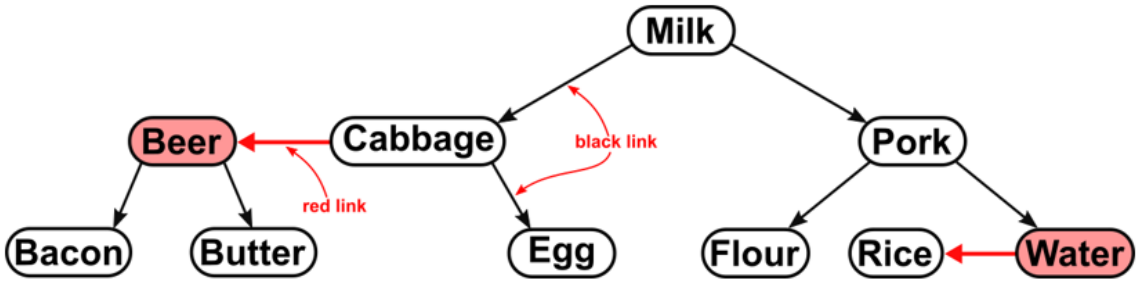


Figure 3.13 red-black containing the same keys of our example. Red-black trees are one of the simplest implementations of 2-3 trees. A red-black BST is like a regular BST, except that the links between nodes can be of two different kinds: red links, and black links. Red links connect keys that, in the corresponding 2-3 tree, would belong to the same 3-node. Black links, instead, would be the actual links of a 2-3 tree.
There are 2 constraints:
1) No node has two red links connected to it (either in- or